

36-662 Final Report (Team Mile High Accuracy)

Pramit Vyas, Dominick Robinson

April 29, 2026

1 Introduction

Flight delays are experienced by travelers across the world for various reasons and can seem arbitrary. But in many ways travelers can prepare ahead if they can predict if their flight will be delayed at all prior to the takeoff of the flight. So we attempt to answer the question: "Can we predict if a flight will be delayed just using information available before the flight takes off?"

1.1 Data

For this, we will use data from the Bureau of Transportation Statistics who has collected, which includes every flight departing from or coming to Pittsburgh (PIT) from the years 2023 and 2024. In addition, we also use some of the flights from 2025. In 2023, we observed 66532 flights coming into or departing from PIT. In 2024, we observed 76475 flights either coming into or departing from PIT. From the 2025 data we only used a subset of 16324 flights randomly decided throughout the year. We also made predictions on a subset of 8687 flights from 2025.

1.1.1 Federal Holidays

People who can travel tend to travel more when they don't have work. People are more likely to not have work on holidays. Therefore, it follows that people may travel more on holidays. Alternatively, people are less likely to travel for work on days they don't work, and more likely to stay home to spend time with family on days they don't work. In either case, we felt that holidays might be a promising covariate for our models.

Figure 1 shows that, at least visually, Federal holidays did not seem to have a higher proportion of delays than other days.

1.1.2 Weather

Bad weather, particularly wind and rain, are causes of flights to be delayed. Pittsburgh, in particular, often has windy and especially rainy weather. We wanted to include historical weather forecasts in our models because accurate forecasts of windy and/or rainy weather would predict flights being delayed.

However, in our searching, we could not find historical weather forecasts. As a proxy, we used historical ASOS weather data from Iowa State University's Iowa State Mesonet. The data included many variables, including hourly precipitation (inches) and hourly wind (knots).

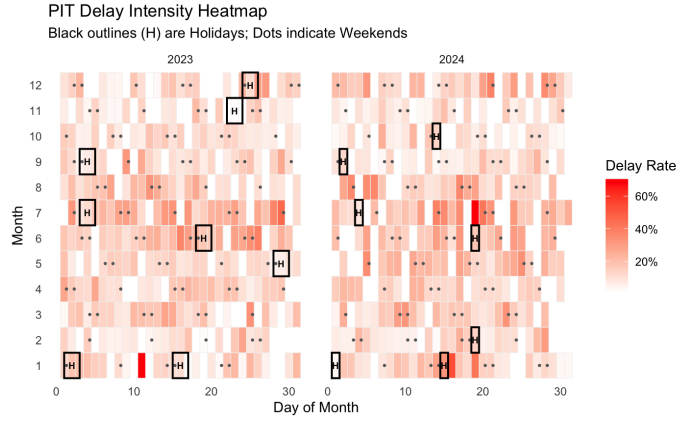


Figure 1: Tiled heatmap of proportion of cancellations by day, month, and year. Dotted tiles indicate weekends, while outlined tiles indicate federal holidays. Visually, federal holidays do not appear to be particularly

2 Exploration

We split our data into training, validation, and testing data. Training and validation data were randomly sampled from the 2023 and 2024 flight data, while the testing data consists solely of the 2025 flight data.

The training data includes 47,474 on-time flights and 8,950 delayed flights.

We also looked at proportion of flights delayed by carrier, because each carrier is likely to have their own logistical challenges for routing flights. Table 2 and Table 1 show the top 5 most and least delayed carriers, respectively, among carriers with more than 100 total flights in the training data. Clearly, the probability of delay varies by carrier, so we should include carrier as a categorical covariate in the models we make.

Table 1: Top 5 Most Delayed Carriers

Carrier Code	Prop. Delayed	Total Flights
F9	0.307	456
NK	0.262	4,263
B6	0.216	1,572
G4	0.214	1,817
AA	0.208	8,087

Table 2: Top 5 Least Delayed Carriers

Carrier Code	Prop. Delayed	Total Flights
YX	0.099	13,376
DL	0.118	4,029
OO	0.133	1,860
9E	0.147	1,831
UA	0.166	3,833

We also investigated how the proportion of flights delayed varies by hour of day for

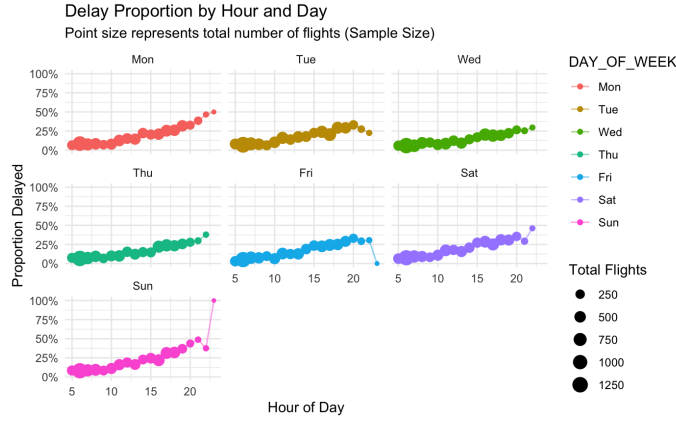


Figure 2: Delays are more common later in the day and on Mondays and Fridays.

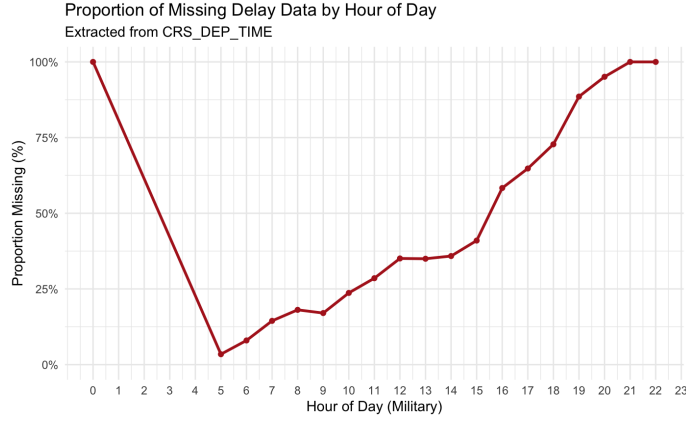


Figure 3: Proportion of missing values in the delay column monotonically increases over time of day, validating the claim in the project handout about the nature of the 2025 flight data split.

each day of the week. Figure 2 shows the proportion of flights delayed by time of day for each day of the week. On average, and for all days of the week, the proportion of delays increases throughout the day, which makes sense considering that delays earlier in the day may propagate and cause later flights to also be delayed. Clearly, we should try to incorporate early-day delays and time of day into our model in some fashion. Additionally, Mondays and Sundays seem to have more delays than the other days, particularly as time increases. Therefore, we should also incorporate day of the week as a categorical covariate in our model.

Finally, to verify that the test data that we needed to generate predictions for was split by time of day, we plotted the proportion of missing values in the `DEP_DELAY15` column, shown in Figure 3. The monotonically increasing plot suggests that this indeed case.

3 Supervised Analysis

For making the predictions we started with the set of all the features that we deemed could have some impact on our final model, these include Quarter, Month, Day of

Month, Day of Year, Distance, CRS Departure time, and CRS elapsed time.

3.1 Feature engineering

We had a few ideas for features that we wanted to include to help predict flight delays.

3.1.1 Holidays

We used the R package `tis` to get the dates of Federal holidays, and added binary variables into our data for each flight for whether or not that date is a Federal holiday.

3.1.2 Weather

We obtained historical ASOS hourly precipitation (inches) and hourly wind (knots) for PIT from Iowa State University's Iowa State Mesonet. To prevent data leakage, we lagged both by one hour.

3.1.3 Previous Flights Information

We postulated that it would be important to know how behind the previous flight is running for the same aircraft given that the next flight is relatively soon. We wanted to feature engineer a variable for this. So we used all of the data provided (including the guess data, more later on why this is ok) to calculate a variable that looks at the previous instance of the same tail number (unique identifier for a specific flight), and checks if its departure has been delayed at all. If it has been delayed then it checks if the previous flight was supposed to land within the previous 3 hours of the flight of interest's estimated departure time. If it was, then we tracked the amount of time it was delayed as a numeric variable. We call this variable last flight delay. We experimented with storing this as a factor, but preliminary modeling and intuition told us that storing this as a continuous numeric variable allows for greater flexibility and more information extracted. We also created a variable in the same step that computes if there was a tight turnaround, this indicates if the previous flight is delayed and is it now scheduled to land less than an hour before departure. Including the data that we are guessing on (i.e our test set) in our calculation data is not feature leakage, since we need the full 2025 dataset to be able to view past flight information, otherwise we would not be able to see how much past flight was delayed by. This is not taking any information that wouldn't be available at the time of the flight. We made sure not to include any features that may go into feature leakage such as looking at the actual arrival delay of the last flight, rather just the departure delay, since the arrival delay may not be decided until the time is right for the next flight to depart.

3.2 Baseline Model

Any model we make, we decided had to beat a baseline model of just predicting all flights will not be delayed. This baseline model had a validation error of .1436 and an AUC of .5 as seen in table 3.

3.3 Other Models we tried

Next we tried a logistic regression model. For this logistic regression model, we did some very basic model selection using both intuition and lack of variable importance to just arrive at a basic logistic regression model having the variables for Carrier, Day of Week, Month, Departure Hour, Last Flight Delay, Tight Turnaround Indicator.

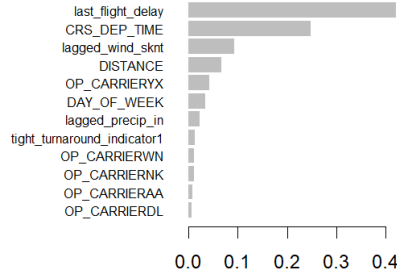


Figure 4: The SHAP variable importance values for the XGBoost Model

And then we also trained another logistic regression model that added outside data including an indicator for holidays, lagged precipitation, and lagged wind speed. Logistic models outperformed the baseline model in terms of AUC scoring .53 and .75 with outside variables, but got worse in terms of test accuracy. This may be due to high variance with logistic regression models in high dimensions and not considering relevant interactions.

Then we tried a random forest model; again we started with the same variables and added the variables pulled from outside sources, but random forest will help lower the variance estimates and account for some of the interactions that would be present in our data. This random forest model did not do great in terms of the validation AUC (hovering around .53 for both models), which could be due to introducing too much bias into the model, and not accounting for the fact that this is a binary problem with the goal being to predict probabilities.

3.4 Final XGBoost model

Finally, we settled on an XGBoost model, after trying the previous models, we wanted a robust model that would mitigate bias and iterate to perform well on the complex and high dimensional nature of our problem by also accounting for mitigating variance. We also tended to really under perform on specific values especially late in the date with our random forest models, as the error was all the way up to .3 for 8pm flights. So since XGBoost weights some of the misclassified samples more, we wanted to use this. We refined the model through an exhaustive grid search paired with 5-fold cross-validation to ensure the results generalized beyond our training set. The winning configuration utilized a learning rate of 0.1 and a max depth of 9, allowing the model to capture intricate interactions without spiraling into overfitting. By subsampling 80 percent of both rows and columns (subsample and colsample bytree), we added the necessary random noise to stabilize the model. This tuning helped lower the error even further as seen in table 3, the XGBoost achieves the best Validation AUC at .77 and the best validation error rate at .14.

Based on figure 4, we can see that the model heavily looks at the variables that relate to the previous flight delays. The dominant relationship is centered on last flight delay, which confirms that the model is seeing the "cascading effect" where a single delayed leg impacts the rest of a tail number's schedule. This predictor carries nearly double the weight of any other variable. XGBoost builds its predictions by

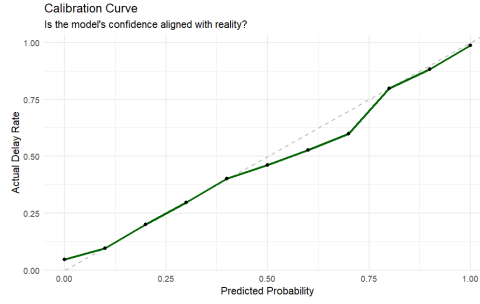


Figure 5: Calibration Plot

looking at how these features interact at different branch depths, so it isn't just seeing CRS DEP TIME alone as a factor but actually its looking at a threshold where the probability of delay shifts as the day progresses. By focusing on these high-gain features, the model creates a predictive logic that moves beyond simple averages and instead targets the specific operational bottlenecks that lead to late-day failures.

4 Analysis of Results

Table 3 show the train, validation, and test errors and AUC's for each model we trained. We trained each model using the training data, selected the "best" model as the model with the lowest validation AUC, and estimated our model performance on the missing 2025 data to be the test AUC. The baseline XGBoost model had the lowest validation error and highest validation AUC of all models we tested, so we selected it as our best model.

Interestingly, none of the models we tested had a lower test error than the baseline "Always On-Time" model. Despite this, we are confident that our selected XGBoost model will outperform (or perform at least as good as) any of the other models because the 2025 data we tested on is a subset of the complete 2025 data featuring a higher proportion of morning flights than the complete 2025 data, based on the split of the data as described in the project handout. This bias in the test data used in the model might explain why the test errors were generally much lower than the validation errors, particularly for the baseline "Always On-Time" model, as earlier flights are less likely to be delayed than later flights.

4.1 What our model did well vs not well

Our model overall is well calibrated, meaning that the predicted probability aligns well with the actual probability of a plane being delayed (this is measured by taking buckets of predicted probabilities and looking at their true probabilities) as seen in figure 5

Even though overall it did well on doing well at predicted probabilities overall, it performs better or worse looking at different subsets of data. Looking at figure 6, we can see that it does very well at predicting for Republic Airlines (YX), Skywest (OO), and American Eagle (OH) seemingly doing better with airlines that are local subsidiaries of bigger carriers. It does poorly at predicting more national brands such as Spirit (NK), B6 (Jetblue) and American (AA). This makes sense as these airlines have a lot of variability in what could cause them to be delayed and a more centralized system meaning delays are handled at a national level, making it more variable. Also looking at figure 7, we can see that the XGBoost model performs

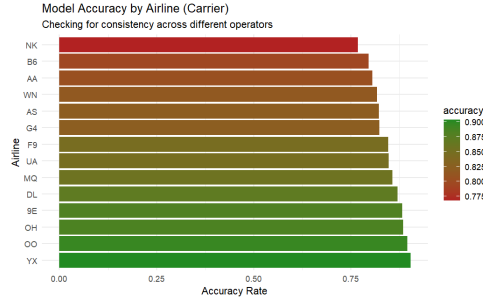


Figure 6: XGBoost Model Performance by Airline

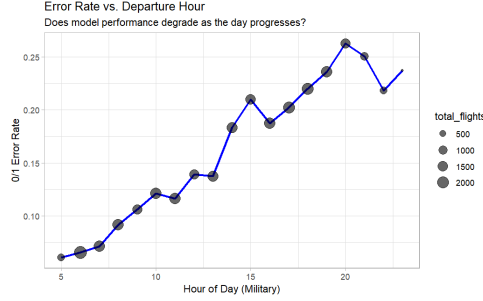


Figure 7: XGBoost Model performance by time of day

much better on flights earlier in the day, and performs poorly on flights as the day progresses up until 8pm. This indicates that we may be missing even more information about backed up flights that could be done in future work.

4.2 What if we wanted to predict the delay minutes instead?

We settled on XGBoost as our model of choice. Like most tree-based models, it has the ability to predict continuous variables. At first, we can try simply swapping the response variable from `DEP_DELAY15` to `DEP_DELAY` and examine its performance. We can also try using some of the other formulas/models we tested. For example, the weather variables may be more useful for predicting slight delays less than 15 minutes that were not captured in the `DEP_DELAY15` variable.

4.3 What if we cared about a trade-off for missing a flight and waiting around for a delayed flight?

In this scenario, we would optimize over a different loss function. In particular, we could train the continuous flight delay (in minutes) predictive model, but optimize over a loss function with formula in Equation (1):

$$L(a, d) = \begin{cases} r(d - a) & \text{if } a \leq d \\ C & \text{if } a > d \end{cases} \quad (1)$$

This answer depends on the ratio of C to r , as relatively high C rewards a model that "plays it safe" because missing a flight (i.e., predictions underestimate flight delay) incurs relatively high penalty, and relatively high r rewards a model that doesn't "play it safe" because waiting around (i.e., predictions overestimate flight delay) incurs relatively high penalty.

4.4 Future Work

One limitation in our feature engineering is that we did not have access to historical forecast data to train on. We used 1-hour-lagged historical weather as a proxy, assuming that wind and rain conditions generally persist from hour to hour. However, if this is not true, then this proxy is not an effective. Future work should try to obtain historical forecast data or even train a forecasting model on the historical data to predict future wind and rain conditions to include as a covariate in our model.

5 Appendix

Table 3: Model Performance Comparison: AUC and Error Rate

Model	Configuration	Training	Validation	Test
<i>AUC (Ranking Power)</i>				
Baseline	Always On-Time	0.5	0.5	0.5
Logistic	Initial	0.5380	0.5294	0.5097
Logistic	Improved	0.7569	0.7520	0.6973
Random Forest	Old	0.5403	0.5307	0.5112
Random Forest	New Variables	0.5354	0.5260	0.5042
XGBoost	Baseline	0.7956	0.7704	0.7141
XGBoost	Weather/Holidays	0.7871	0.7664	0.7158
XGBoost	New Best Params	0.8306	0.7682	0.7146
<i>0/1 Error Rate</i>				
Baseline	Always On-Time	0.1586	0.1436	0.1137
Logistic	Initial	0.1804	0.1814	0.1364
Logistic	Improved	0.1556	0.1584	0.1250
Random Forest	Old	0.1804	0.1814	0.1364
Random Forest	New Variables	0.1804	0.1814	0.1364
XGBoost	Baseline	0.1533	0.1431	0.1247
XGBoost	Weather/Holidays	0.1516	0.1554	0.1247
XGBoost	New Best Params	0.1457	0.1560	0.1242